

Sliding Window & Rate Limiting

5 interview-style practice problems built around the same core pattern: time windows, queues/deques, expiration, duplicates, and space optimization.

Basic sliding window	1	Deque + expiration
Per-key limiter	2	Hash map + deque
High-volume duplicates	3	Counts / buckets
Out-of-order timestamps	4	Sorted state
Record + query API	5	Fast queries

Before coding: define the exact window boundaries, decide what gets stored, and state your target time and space complexity.

Problem 1: Sliding Window Event Counter

CORE PATTERN

Prompt

Design an in-memory event counter. Implement EventCounter with a method `record(timestamp: int) -> int` that records an event at the given second and returns the number of events in the last 30 seconds, meaning the inclusive interval `[timestamp - 29, timestamp]`. Timestamps are non-decreasing.

Example / constraint

Input timestamps: 1, 4, 4, 10, 31

Expected returned counts: 1, 2, 3, 4, 4

Follow-up

Follow-up: What changes if the caller can ask for `count(timestamp)` without adding an event? How would you keep the operation efficient?

What this is testing: Deque / queue, removing expired timestamps, amortized $O(1)$ thinking, inclusive time windows.

Your approach

Problem 2: Login Attempt Rate Limiter

INTERVIEW STANDARD

Prompt

Design a per-user login rate limiter. Implement `allow(user_id: str, timestamp: int) -> bool`. A user may make at most 5 attempts in any 60-second window. The current attempt should count toward the limit. Return `True` when the attempt is allowed and `False` when it must be rejected. Calls for different users are interleaved.

Example / constraint

For user A at times 10, 20, 30, 40, 50, 55: the first five are allowed; the sixth is rejected. A request at 70 should be evaluated against the window [11, 70].

Follow-up

Follow-up: Should a rejected request be stored? Explain what you choose and why. Then give the time and space complexity.

What this is testing: Map from user to deque, per-key state, expiration rules, whether rejected events consume capacity.

Your approach

Problem 3: Millions of Events at the Same Timestamp

SPACE OPTIMIZATION

Prompt

A telemetry service receives millions of events per second. Implement RateLimiter with `record(timestamp: int) -> bool`, allowing at most 100,000 events in the last 60 seconds. Duplicate timestamps are extremely common, so storing one integer per event can use too much memory.

Example / constraint

A burst may contain 2,000,000 events all stamped with timestamp 500. Your design should not need 2,000,000 separate queue entries just to represent that second.

Follow-up

Follow-up: Replace individual timestamps with (timestamp, count) entries or a circular bucketed array. Explain how you evict old counts and how you handle a timestamp that appears multiple times.

What this is testing: Run-length / count compression, bucketed sliding windows, bounded memory $O(W)$, where W is the time-window width.

Your approach

Problem 4: Out-of-Order Events

HARDER VARIANT

Prompt

Design an in-memory API gateway limiter. Events may arrive out of order. Implement `record(timestamp: int) -> bool` so that at most 20 requests are allowed in the last 10 seconds. Unlike the basic problem, a request at time 100 can arrive after one at time 105.

Example / constraint

Arrival order: 100, 105, 102, 99, 108. The limiter must evaluate each event using event time, not arrival order.

Follow-up

Follow-up: A plain deque only works cleanly when timestamps are non-decreasing. What data structure would you use here? Discuss the extra cost compared with the original problem.

What this is testing: Why monotonic timestamps matter, sorted structures / heaps / trees, tradeoff from $O(1)$ amortized toward $O(\log n)$.

Your approach

Problem 5: Sliding Window with a Query API

DESIGN + REASONING

Prompt

Build a monitoring component that supports two operations:

`record(timestamp: int)` - add one event at that second.

`count(timestamp: int) -> int` - return how many events occurred in the last 300 seconds, using the inclusive window `[timestamp - 299, timestamp]`.

Timestamps are non-decreasing, but calls to count do not add events.

Example / constraint

Operations can be mixed: `record(10), record(10), count(10), record(50), count(309), record(310), count(310)`.

Follow-up

Follow-up: Suppose queries can ask for arbitrary window sizes, such as 10 seconds, 60 seconds, or 300 seconds. How would you redesign the stored information so that queries stay fast without storing every event individually?

What this is testing: Separating updates from queries, retaining enough history, prefix sums / timestamp counts, choosing space for the query workload.

Your approach

Practice Checklist

Use this checklist after solving each problem aloud.

1. Can I state the exact time window, including whether both endpoints are included?
2. Can I explain why old entries are removed and why each entry is removed only once?
3. Can I justify the amortized $O(1)$ deque solution instead of just naming it?
4. Can I handle duplicate timestamps without storing one object per event?
5. Can I explain how the design changes for multiple users or multiple keys?
6. Can I identify what breaks when timestamps arrive out of order?
7. Can I give both time and space complexity in terms of the window size and event volume?

Target interview habit: Start with the simplest correct deque solution. Then improve only when the follow-up introduces a concrete problem such as duplicates, huge bursts, multiple users, or out-of-order data.